

Homework3——24-point Game

Task1

关于四个数字的表达式括号格式，根据卡特兰数共计有

$$C_4 = \frac{1}{4+1} \binom{2 \times 4}{4} = \frac{1}{5} \binom{8}{4} = \frac{1}{5} \times \frac{8 \times 7 \times 6 \times 5}{4 \times 3 \times 2 \times 1} = 14$$

对于24点数的问题，卡特兰数计算的是二叉树结构形态，区分左右节点，但实际上：

$$a + b = b + a$$

$$a * b = b * a$$

故而，很多的结构被合并了，一共只有五种情况。我采用了遍历+逆波兰表达式+单栈的模式算法完成24-point game。

Implementation1: 遍历

- 引入itertools工具完成排列组合的遍历。

```
import itertools
from collections import deque
```

- 计算后缀表达式

```
def rpn_solve(tokens):
    stack=[]
    for token in tokens:
        if token in "+-*/":
            b=stack.pop()
            a=stack.pop()
            if token=="+":
                stack.append(a+b)
            elif token=="-":
                stack.append(a-b)
            elif token=="*":
                stack.append(a*b)
            elif token=="/":
                stack.append(a/b)
        else:
            stack.append(int(token))
    return stack.pop()

tokens = input().split()
print(rpn_solve(tokens))
```

- 循环遍历生成表达式

```
def tfgenerate(nums):
    ops=["+", "-", "*", "/"]
    expressions=[]

    for prob in itertools.permutations(nums):
```

```

a,b,c,d=prob
for op1,op2,op3 in itertools.product(ops,repeat=3):
    expressions.append([str(a), str(b), op1, str(c), op2, str(d), op3])
    expressions.append([str(a), str(b), str(c), op2, op1, str(d), op3])
    expressions.append([str(a), str(b), str(c), op2, str(d), op3, op1])
    expressions.append([str(a), str(b), str(c), str(d), op3, op2, op1])
    expressions.append([str(a), str(b), op1, str(c), str(d), op3, op2])

return expressions
print(tfgenerate([1,2,3,4]))

```

- 判断是否合理

```

def tfsearch(nums):
    expressions=tfgenerate(nums)
    true_ex=[]
    for expression in expressions:
        if abs(rpn solve(expression))<1e-6:
            true_ex.append(expression)

    return true_ex

```

- 后缀转中缀

```

def rpn2in(tokens):
    stack=[]
    for token in tokens:
        if token in "+-*/":
            a=stack.pop()
            b=stack.pop()
            stack.append("(" + b + token + a + ")")
        else:
            stack.append(token)

    return stack.pop()

```

- 测试

```

nums = [9, 2, 10, 4]
valid_rpn = tfsearch(nums)
if valid_rpn:
    for rpn in valid_rpn:
        infix = rpn2in(rpn)
        print(f"{infix}=24")
else:
    print("No solution found")

```

- 结果

```

(9-((10/2)+4))=24
((9-(10/2))-4)=24
(((9-4)*2)-10)=24
(9-(4+(10/2)))=24

```

```

((9-4)-(10/2))=24
((2*(9-4))-10)=24
(2-(10/(9-4)))=24
(2+(10/(4-9)))=24
((2*(4-9))+10)=24
(10-((9-4)*2))=24
((10/(9-4))-2)=24
(10-(2*(9-4)))=24
(((10/2)-9)+4)=24
((10/2)-(9-4))=24
(10+(2*(4-9)))=24
(((10/2)+4)-9)=24
((10/2)+(4-9))=24
(10+((4-9)*2))=24
((10/(4-9))+2)=24
(((4-9)*2)+10)=24
((4-9)+(10/2))=24
(4-(9-(10/2)))=24
((4+(10/2))-9)=24
(4+((10/2)-9))=24

```

Test

我用codegeex生成了100set和10000set的脚本测试，具体代码请见 [Self-24p.ipynb](#)

发现10000set的好事时间过长，说明该算法效率较低，时间复杂度虽然是O(1)，但每次计算都要计算7680次表达式求值。故而，我转向了解决24点数游戏的标准解法，即在codegeex生成的用来评估遍历算法的准确度时，所使用的dfs算法。

Implementation2: DFS

dfs递归解决问题的关键是将原本四个数字进行互相运算的问题转化成更小的问题，我们先通过从nums备选数字抽取两个数字做加减乘除的任意运算，注意：一共六种方式：

$$\frac{a}{b}, \frac{b}{a}, a + b, a * b, a - b, b - a$$

随后，将剩余两个数字加上新得出六个数字中的一个，进行递归。接下来，两个。直到剩余一个的时候验证nums里面剩余的值是否为24，即能成功求解。dfs会尽可能深的找到分支，如果无法得到24，则回溯到上一个节点再选择另一个数字求解。

```

def solve_oracle(nums):
    if len(nums)==1:
        return abs(nums[0]-24)<1e-6

    for i in range(len(nums)):
        for j in range(len(nums)):
            if i==j:
                continue
            next_nums=[nums[k] for k in range(len(nums)) if k!=i and k!=j]

            a, b=nums[i], nums[j]

            results=[a+b,a-b,b-a,a*b]
            if abs(b)>1e-6:

```

```
        results.append(a/b)
    if abs(a)>1e-6:
        results.append(b/a)
    for res in results:
        if solve_oracle(next_nums+[res]):
            return True

    return False
```

具体的测试集见代码 [24p-dfs.ipynb](#)

Analysis

dfs算法的速度远远超过普通遍历，速度约在7倍左右，原因是dfs深度优先搜索有条件地停止对一些无效或不可能得到解的路径的探索，即剪枝，从而大大减少了计算量。